

# ECE501-2025-ProjectNo16-GroupNo4-Weekly Report-2

Hetvi Shah(AU2340101)      Manushree Patel(AU2340014)  
Twisha Pate(AU2340056)      Sanya Shah(AU2340259)

October 2025

## Abstract

Massive amounts of data are produced by medical imaging, necessitating effective transmission and storage without sacrificing diagnostic value. The theoretical underpinnings of picture partitioning for lossless JPEG compression-based compact encoding are presented in this research. Redundancy is decreased and important visual details are maintained by splitting an image into several rectangular segments and encoding each one separately. High compression ratios appropriate for medical applications where precision is crucial are the goal of the method.

## 1 Introduction

There is an exponential growth in the technologies of medical imaging. Technologies like x-rays, MRI and CT scans have led to a rise in the amount of images generated daily. Since these medical images frequently include important diagnostic data, they must be sent and stored without sacrificing image quality or data. However, effectively managing massive amounts of data while preserving diagnostic accuracy presents significant hurdles for medical organizations. Image compression methods are used to solve this problem by lowering the file size while maintaining necessary data. Among several tactics, picture partitioning for compact encoding has proven to be a successful method for preserving the integrity of medical images while maximizing compression speed.

**Keywords:** Medical Imaging, Image Compression, Lossless Compression, JPEG Compression, Picture Partitioning, Compact Encoding, Data Transmission, Data Storage, Diagnostic Accuracy, Redundancy Reduction, Rectangular Segmentation, High Compression Ratio, Image Quality Preservation, MRI, CT Scan, X-ray

## 2 Early Results

According to the project definition and the approach in the paper, we have implemented the project in the following steps:

1. Load an image and convert it to grayscale.
2. Detect important regions (ROI) based on brightness.
3. Divide the image into  $C$  uniform rectangles.
4. Encode each rectangle:
  - **ROI blocks:** keep original.
  - **RONI blocks:** apply blur and quantization  $\rightarrow$  lossy compression.
5. Visualize the original image, block layout, and compressed image.
6. Save the compressed image and report the compression ratio.

The code implemented is:

Listing 1: ROI-Preserving Image Compression Python Code

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import os
5
6 # ----- ROI Detection (intensity-based) -----
7 def detect_roi(image, thresh_factor=0.5):
8     mean_intensity = np.mean(image)
9     roi_mask = (image > mean_intensity * (1 + thresh_factor))
10     roi_mask = cv2.medianBlur(roi_mask, 9)
11     return roi_mask
12
13 # ----- Uniform Partition -----
14 def uniform_partition(image, n_blocks):
15     H, W = image.shape[:2]
16     num_blocks_side = int(np.sqrt(n_blocks))
17     w = W // num_blocks_side
18     h = H // num_blocks_side
19     regions = [(x * w, y * h, w, h)
20               for y in range(num_blocks_side)
21               for x in range(num_blocks_side)]
22     return regions
23
24 # ----- ROI-preserving Compact Encoding -----
25 def compact_encode_roi_strict(image, regions, roi_mask):
26     encoded = np.zeros_like(image)
```

```

27     for (x, y, w, h) in regions:
28         region = image[y:y+h, x:x+w]
29         roi_ratio = np.mean(roi_mask[y:y+h, x:x+w])
30         if roi_ratio > 0.05:
31             region_encoded = region.copy()
32         else:
33             blurred = cv2.GaussianBlur(region, (9,9), 0)
34             region_encoded = np.uint8(np.round(blurred / 32)
35                                         * 32)
36             encoded[y:y+h, x:x+w] = region_encoded
37     return encoded
38
39 if __name__ == "__main__":
40     img_path = "/content/Y1.jpg"
41     C = 100
42     img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
43     roi_mask = detect_roi(img, thresh_factor=0.3)
44     regions = uniform_partition(img, C)
45
46     img_rect = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
47     for (x, y, w, h) in regions:
48         cv2.rectangle(img_rect, (x, y), (x + w, y + h), (0,
49             0, 255), 1)
50
51     lossy_encoded = compact_encode_roi_strict(img, regions,
52         roi_mask)
53
54     plt.figure(figsize=(15,5))
55     plt.subplot(1,3,1)
56     plt.imshow(img, cmap='gray')
57     plt.title("Original Image")
58
59     plt.subplot(1,3,2)
60     plt.imshow(img_rect[...,:-1])
61     plt.title(f"Uniform {C} Rectangles (ROI-aware)")
62
63     plt.subplot(1,3,3)
64     plt.imshow(lossy_encoded, cmap='gray')
65     plt.title("ROI-Preserved Lossy Encoding")
66
67     plt.tight_layout()
68     plt.show()
69
70     cv2.imwrite("roi_strict_lossy_output_q20.jpg",
71         lossy_encoded,
72         [int(cv2.IMWRITE_JPEG_QUALITY), 20])
73
74     original_size = os.path.getsize(img_path) / 1024
75     compressed_size = os.path.getsize("
76         roi_strict_lossy_output_q20.jpg") / 1024

```

```
72     compression_ratio = original_size / compressed_size if  
       compressed_size else 0  
73  
74     print(f"\nOriginal Size: {original_size:.2f} KB")  
75     print(f"Compressed Size: {compressed_size:.2f} KB")  
76     print(f"Compression Ratio: {compression_ratio:.2f}")  
77     print("ROI fully preserved; RONI compressed.")
```

The results of the code are:

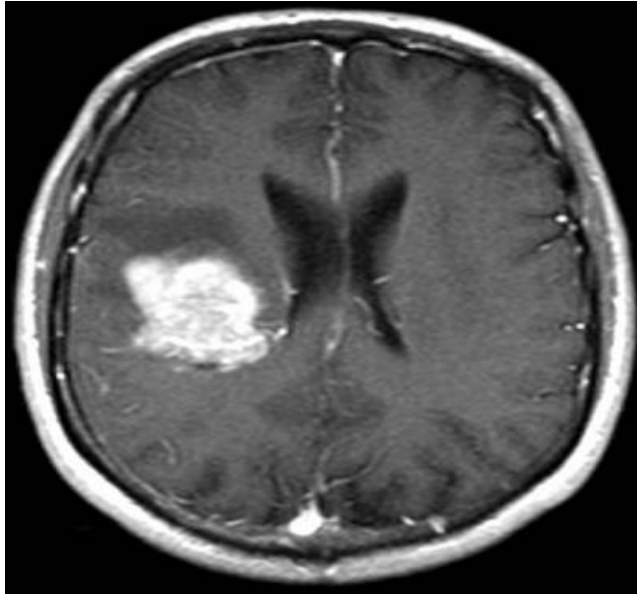


Figure 1: Original image.

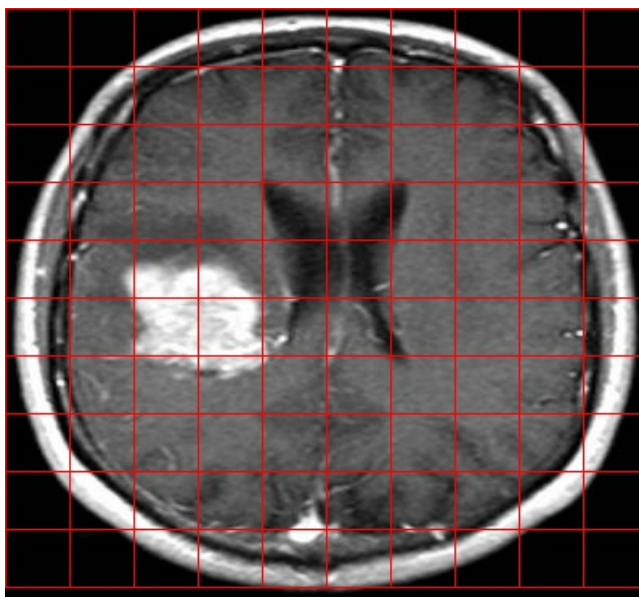


Figure 2: Partioned image.

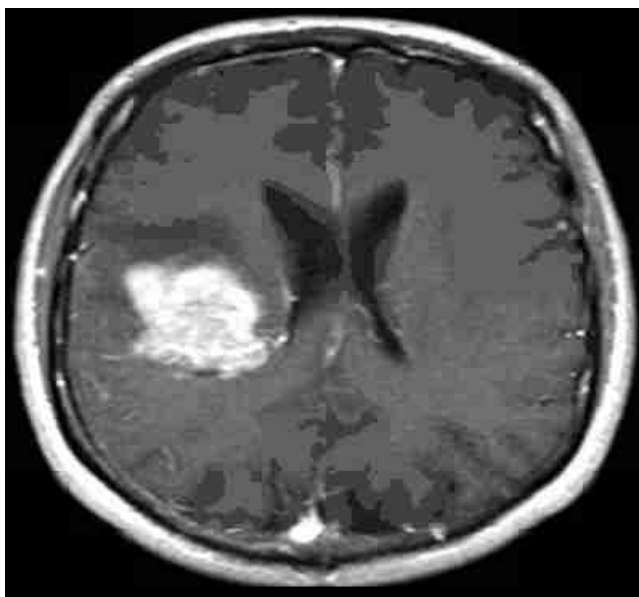


Figure 3: Compressed image.

The size of original image is 18.80 KB and after compressing it is 7.20 KB and compression ratio is 2.61.

### 3 References

Ungureanu, V.-I., Negirla, P., Korodi, A. (2024). \*Image-compression techniques: Classical and “region-of-interest-based” approaches presented in recent papers\*. \*Sensors\*, 24\*(3), 791. [<https://doi.org/10.3390/s24030791>](<https://doi.org/10.3390/s24030791>)

Bairagi, V., Sapkal, A. M. (2013). ROI-based DICOM image compression for telemedicine. Indian Academy of Sciences, 38(1), 123–131. <https://www.ias.ac.in/public/Volumes/sadh/038/01/0123-0131.pdf>